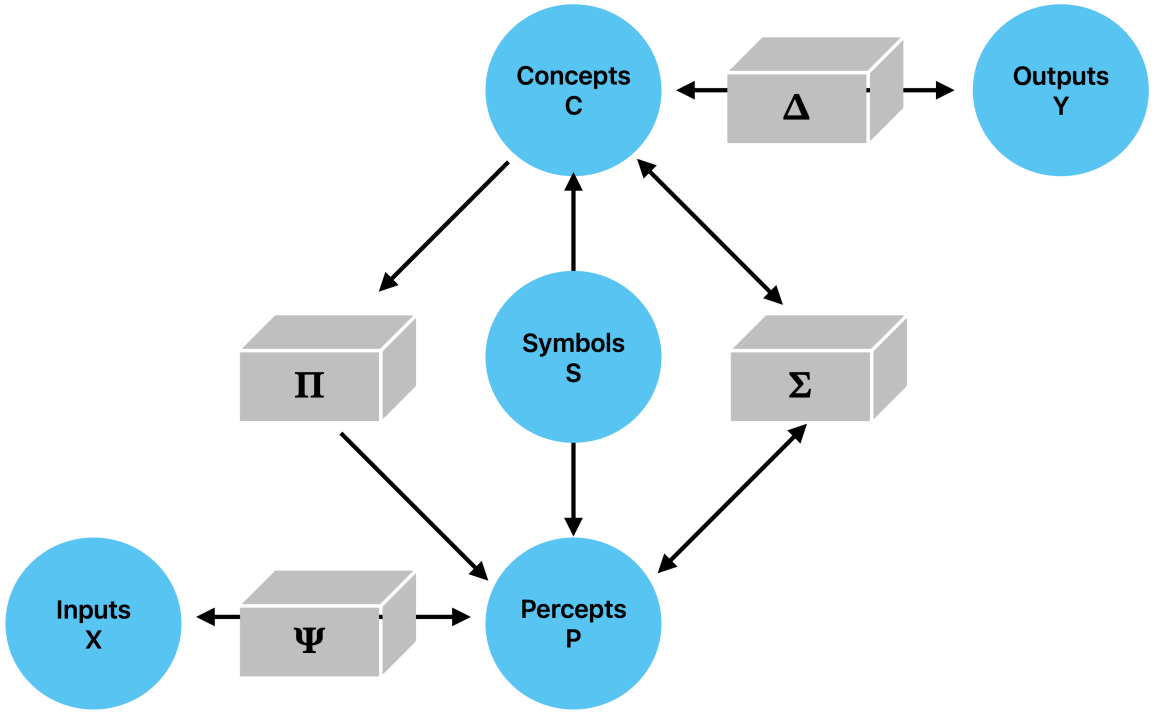
Basic Model

Contents

BasicModel

Basic Model of Cognition



The basic model of cognition relies on conceptual hyperplanes and perceptual prototypes to synthesize and analyze the input space. It uses a high-dimensional embedding to characterize mental space and integrates symbolic computation. Its three major operations are intersection (which forms percepts from concepts), union (a bidirectional mapping between concepts and percepts), and equality (where symbols are elements that map across perceptual and conceptual domains).

Overview

BasicModel is a parameterized neural architecture with three independent levers:

- **Ergodic** — adaptive bias-variance control via a gradient energy sensor (not gradient descent)

- **Certainty** — per-neuron certainty tracking; neurons graduate from exploration to exploitation independently
- **Reversible** — bidirectional training: forward prediction + backward reconstruction in a single optimizer pass

Model configurations are specified in XML and can be compared side-by-side. See doc/Architecture.md for the full mathematical treatment.

Files

File	Description
bin/BasicModel.py	Main entry point: model factory, training loop, comparison plots, HTML report
bin/Model.py	Layer library: SigmaLayer, PiLayer, ErgodicLayer, LinearLayer, spaces
bin/embed.py	Word vector training: CBOW/SBOW with negative sampling, `WordVectors` (gensim-compatible `.kv`
bin/SigmaPi.py	Standalone demo of the SigmaPi network solving XOR
data/	XML model configurations
doc/Architecture.md	Algorithm details: Sigma/Pi layers, ergodic exploration, gradient energy sensor
doc/Params.md	Full XML parameter reference
doc/Training.md	Embedding pretraining, CBOW/SBOW, masked prediction, `<trainEmbedding>` modes
doc/Installation.md	Setup, Makefile targets, train.py options, remote training
test/	Unit tests

Quick Start

```
# Set up virtual environment
make install

# Run a single model
make simple          # data/simple.xml
make ergodic         # data/ergodic.xml

# Compare two models side-by-side
make compare         # defaults: data/simple.xml vs data/ergodic-only.xml
make compare XML1=data/simple.xml XML2=data/ergodic.xml

# Run tests
make test

# Generate PDF documentation
make doc_pdf
```

XML Configuration

Models are configured via XML files in `data/`. Training and data parameters live in nested sub-elements:

```
<model>
  <architecture>
    <modelType>simple</modelType>   <!-- simple | lm | embedding -->
    <ergodic>false</ergodic>
    <certainty>false</certainty>
    <reconstruct>NONE</reconstruct>  <!-- NONE | symbols | output | both -->
    <maskedPrediction>NONE</maskedPrediction>

    <data>
```

```
      <dataset>xor</dataset>          <!-- xor | mnist | text | ... -->
    </data>

    <training>
      <numTrials>1</numTrials>
      <numEpochs>20</numEpochs>
      <batchSize>10</batchSize>
      <learningRate>0.001</learningRate>
      <weightsPath>output/BasicModel.ckpt</weightsPath>
      <autoload>true</autoload>
      <autosave>false</autosave>
    </training>
  </architecture>

  <InputSpace> ... </InputSpace>
  <PerceptualSpace> ... </PerceptualSpace>
  <ConceptualSpace> ... </ConceptualSpace>
  <SymbolicSpace> ... </SymbolicSpace>
  <OutputSpace> ... </OutputSpace>
</model>
```

See doc/Params.md for the full parameter reference.

Output

Each run produces an HTML report (timestamped in `output/`) containing:

- **Error per Epoch** — training and test loss curves
- **Accuracy per Digit** — per-class accuracy breakdown

In compare mode, additional overlay plots show combined loss and accuracy across models with color-coded legends.

A Basic Model for Cognition

This essay explores a basic model of human cognition. The possible implementations of such a model create a spectrum of complexity with two theoretical extremes. On the simple end of the spectrum are models such as behaviorism that treat all of cognition as a subjective "black box" about which nothing can be said (as in Figure A). Modern LLMs are example implementations of this paradigm. On the complex end of the spectrum are models that describe numerous and detailed aspects of cognition and their corresponding biological implementation, which also tend to be opaque.

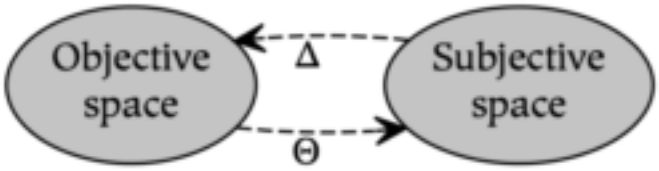


***Figure A*: The behaviorist model analyzes the world in terms of the stimuli (Θ) and responses (Δ) that occur between objective (or environmental) spaces and subjective (or mental) spaces.**

Perhaps the most simple modification of the behaviorist model divides subjectivity into two constituents. Although there are different ways to accomplish this division, one of the most common ways distinguishes conceptual and nonconceptual content. In Dual Process Theory, for example, this distinction creates two systems called System 1 and System 2. In ACT-R, a similar distinction differentiates declarative and procedural knowledge. In theoretical linguistics, this fundamental dividing line differentiates syntax from

semantics. However, despite the prevalence of these parallel distinctions within cognitive science, they do not serve as a unifying theme. As Johnathan Evans summarizes in a discussion of Dual Process Theory:

The distinction between two kinds of thinking, one fast and intuitive, the other slow and deliberative, is both ancient in origin and widespread in philosophical and psychological writing. Such a distinction has been made by many authors in many fields, often in ignorance of the related writing of others. [Evans & Stanovich, 2013].

A novel diagram representing this basic model of cognition is presented here to unify this disparate terminology. Subjective space is divided into sensory and conceptual spaces to approximate the conceptual/nonconceptual distinction referenced above. As a result, while the perception of a tree in the behaviorist model is characterized only in terms of the response to that perception, the basic model of cognition described below further analyzes subjective experience in terms of the bottom-up concepts formed in response to sensation ($\mathbf{\Phi}$) and the top-down visualization of those concepts ($\mathbf{\Psi}$).

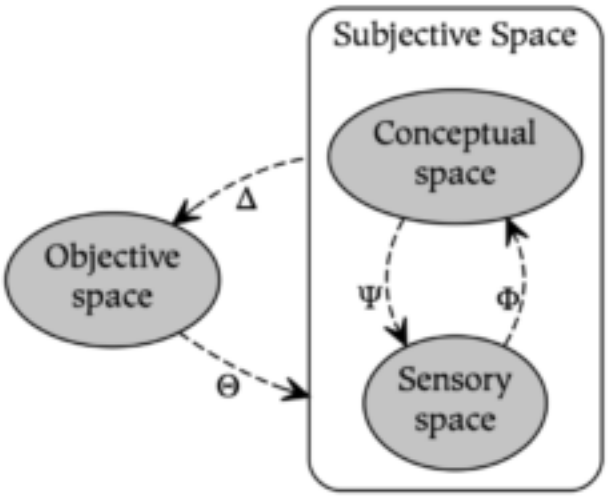


Figure B: Dividing subjective space into sensory and conceptual spaces allows modeling bottom-up *conceptualization* ($\mathbf{\Phi}$) and *top-down visualization* ($\mathbf{\Psi}$).

In Figure B, although sensory and conceptual spaces are depicted as discrete entities, these nodes represent a single space that ranges from increasingly sensory parts to increasingly conceptual wholes. In other words, sensory content becomes *increasingly* conceptual as one moves bottom-up from sensory space to conceptual space, and conceptual content becomes *increasingly* sensory as one moves top-down from conceptual space to sensory space.

A significant limitation of using the model in Figure B, however, is that it only describes symbolic processes. In particular, *conceptualization* ($\mathbf{\Phi}$) and *visualization* ($\mathbf{\Psi}$) are subsymbolic rather than symbolic, and correspond to the relation between parts and wholes. As a result, there is no way within the model of Figure B to capture the notion that the symbol "tree" is a representation of the concept tree, since they are not related to one another as parts or wholes (i.e., symbols are references to what they symbolize). Therefore, the description of symbolic processes requires the addition of two further relations: *symbolization* ($\mathbf{\Xi}$), which creates symbolic references from conceptual content, and *interpretation* ($\mathbf{\Omega}$), which dereferences symbols to activate their associated concepts. These relations allow one to model both the interpretation (or understanding) of the symbol of a tree as well as the symbolization (or naming) of the concept of a tree.

Adding these two relations to the model in Figure B results in the following *basic model of cognition*, which consists of three spaces and six relations:

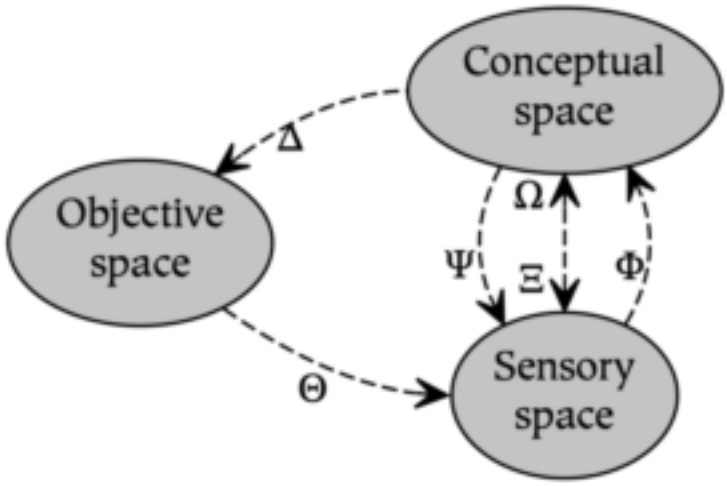
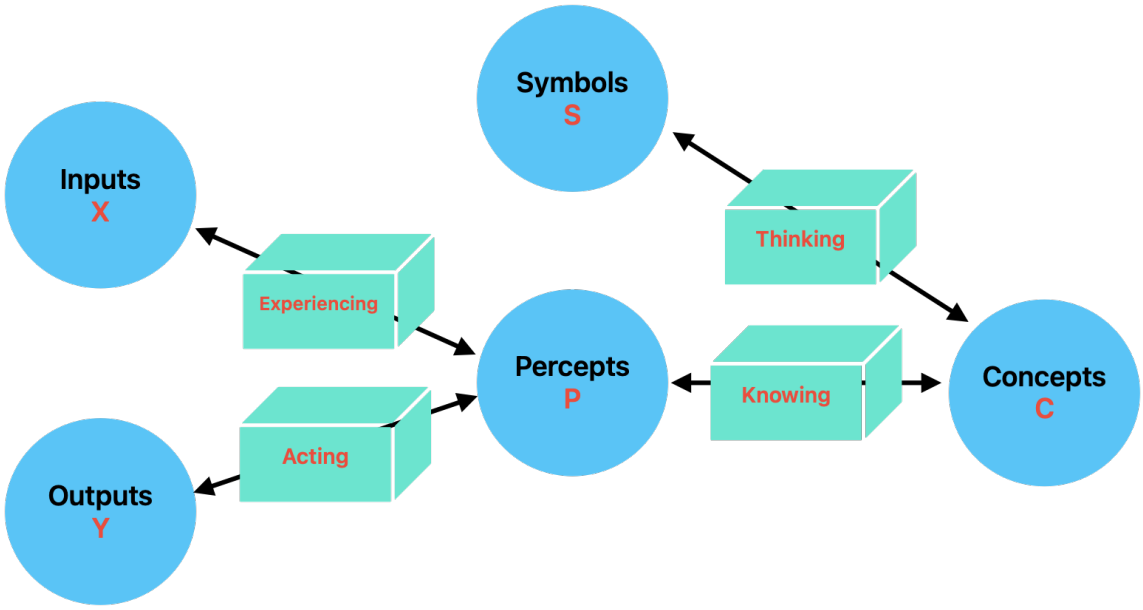


Figure C: The basic model of cognition.

While this model is quite simple, it extends the more basic rubric of Dual Process Theory by adding terminology for both subsymbolic and symbolic operations. One of its key strengths is that it provides a common foundation for a number of existing theories in cognitive science; for example, first-order logic, syntactic hierarchies, and mathematical sets can all be recursively constructed using conceptualization and symbolization (this is illustrated in the book The Whole Part [Rogers, 2020]). In distinction to these formalisms, however, the basic model of cognition also allows one to model animal cognition and other subsymbolic processes.



Architecture

Overview

BasicModel is a bidirectional neural architecture organized as a pipeline of five **spaces**, each implementing a distinct representational transformation:

```
Forward:  InputSpace → PerceptualSpace → ConceptualSpace → SymbolicSpace → OutputSpace
Reverse:  OutputSpace → SymbolicSpace → ConceptualSpace → PerceptualSpace → InputSpace
```

The forward pass transforms raw input into predictions. The reverse pass reconstructs the original input from the symbolic representation. Both directions are trained simultaneously with a single optimizer minimizing a combined loss:

```
totalLoss = (1 - reconRatio) * outputLoss + reconRatio * reconstructionLoss
```

Spaces

Space	Role	Layer Type	Parameters
InputSpace	Lifts raw data into working dimensionality	LiftingLayer	nActive, nDim, nVecto
PerceptualSpace	Multiplicative feature extraction	PiLayer	nActive, nDim, nVecto
ConceptualSpace	Additive abstraction	SigmaLayer	nActive, nDim, nVecto
SymbolicSpace	Information bottleneck	passThrough or learned	nActive, nVectors, pas
SyntacticSpace	Syntactic processing	passthrough (future: grammar)	nActive, nDim (wordI
OutputSpace	Final prediction	LinearLayer	nActive, nDim, nVecto

Dimensions (`nDim`) are not passed to Space constructors; each subclass reads its content dimensionality from `TheObjectEncoding` (e.g., `TheObjectEncoding.perceptDim` for PerceptualSpace). Codebook sizes (`nVectors`) are likewise stored on `TheObjectEncoding` and may differ from the active count (`nActive`); the factory validates `nVectors >= nActive` for every space.

Layer selection depends on `<reconstruct>` and `invertible`:

1. **`reconstruct=NONE`**: Use non-invertible layers (`PiLayer`, `SigmaLayer`) for the forward pass only. No reverse pipeline is created.
2. **`reconstruct=<any>` + `invertible`**: A single invertible layer (`InvertiblePiLayer`, `InvertibleSigmaLayer`) serves both directions, sharing weights.
3. **`reconstruct=<any>` + not `invertible`**: Two layers with separate weights — call `forward()` on one and `reverse()` on the other. This avoids the expressivity limitation where a non-invertible layer's forward pass cannot represent the inverse of another (e.g., PiLayer's product structure is not closed under inversion). **Note:** The reverse path uses a matrix pseudoinverse (`pinv`) which may be numerically unstable due to SVD convergence issues. Setting `<invertible>true</invertible>` avoids this by using shared-weight inversion instead.

Reconstruction Symbols

The symbolic bottleneck can lose information needed for reconstruction. For example, XOR maps 2 inputs to 1 output, but `XOR(0,0)=0` and `XOR(1,1)=0` are distinct inputs producing the same output. A single output symbol cannot reconstruct which input was presented.

To solve this, the `nSymbols` produced by SymbolicSpace are split:

- **`nOutputSymbols = OutputSpace.nActive`** — fed to OutputSpace for prediction
- **`nReconSymbols = nSymbols - nOutputSymbols`** — carried in `end_state` for reconstruction

This is essentially a skip connection through the symbolic bottleneck: an extra channel that preserves information lost in the output projection. Reconstruction symbols receive gradient only from reconstruction loss, never from output loss.

For XOR with `nSymbols=3`, `nOutput=1`: symbol 0 predicts the output; symbols 1-2 carry enough information to distinguish all 4 input patterns, enabling perfect reconstruction.

Single Optimizer with Overlapping Weight Spaces

A critical design choice: the forward and reverse passes share a **single Adam optimizer** that minimizes a combined loss:

```
totalLoss = (1 - reconRatio) * outputLoss + reconRatio * reconstructionLoss
```

This works because the forward and reverse weight spaces **partially overlap**. They are neither disjoint (which would allow independent optimizers) nor identical (which would create destructive interference). Instead, certain layers share weights between directions (e.g., shared embeddings, the symbolic bottleneck itself) while others are direction-specific (e.g., `pi1`/`pi2`, `sigma1`/`sigma2`, `linear1`/`linear2`).

The partial overlap means:

- **Shared weights** receive gradient from *both* output and reconstruction loss, learning representations useful in both directions simultaneously.
- **Direction-specific weights** receive gradient from only their respective loss, specializing freely without interference.
- **No ping-pong**: separate optimizers on overlapping parameters would pull weights in alternating, conflicting directions each step. A single optimizer sees the combined gradient and finds a consistent descent direction.

This resolves the fundamental tension between forward prediction and backward reconstruction in bidirectional architectures — a problem first identified in A.M. Rogers, T.T. Shannon, and G.G. Lendaris, "A comparison of DHP based antecedent parameter tuning strategies for fuzzy control," *Proceedings Joint 9th IFSA World Congress and 20th NAFIPS International Conference*, Vancouver, BC, Canada, 2001, pp. 580-585, doi: 10.1109/NAFIPS.2001.944317. See also `doc/research/` for a local copy.

When `invertible=true`, the overlap is total: a single invertible layer serves both directions, and its weights receive the full combined gradient. The single-optimizer approach handles this case naturally.

Training Loop

Training uses the single Adam optimizer with persistent state (momentum/variance accumulate across epochs):

1. Forward pass: input → prediction + `end_state`
2. Compute `outputLoss` from prediction vs. target
3. Reverse pass: `end_state` → reconstructed input
4. Compute `reconstructionLoss` from reconstruction vs. `end_state`
5. Backpropagate `totalLoss = (1 - reconRatio) * outputLoss + reconRatio * reconstructionLoss`
6. If ergodic: run `paramUpdate()` (gradient energy sensor updates alpha)
7. Optimizer step (embedding params excluded when `trainEmbedding` is `NONE` or `ARLM`)
8. If `trainEmbedding` is `CBOW`, `SBOW`, or `BOTH`: run embedding update step on same sentence

Alpha annealing (ergodic mode): the code-level exploration parameter starts at `1.0` (full exploration) and decays to `0.0` (full exploitation) within the first 5% of epochs via `alpha = max(0, 1 - epoch / warmup)` where `warmup = numEpochs // 20`. Note: the code convention (`alpha=1` means explore) is the inverse of the ergodic math convention (`alpha=0` means explore); layers translate between them internally.

See Params.md for all XML configuration parameters. See Training.md for embedding pretraining, SBOW, masked prediction modes, and the `<trainEmbedding>` control.

Three-File Architecture

Model behaviour is partitioned across three independent artifacts:

File	Contents	Managed by
XML config (e.g. `BasicModel.xml`)	Architecture, hyperparameters, paths	Hand-edited
Embedding artifact (e.g. `BasicModel.kv`)	Word vectors (codebook)	Phase 1: `embed.py`
Weights checkpoint (e.g. `BasicModel.ckpt`)	Model layer parameters (excludes embeddings)	Phase 2: `BasicModel.py`

The `save_weights()` method explicitly filters out embedding parameters (`_emb.weight`) from the checkpoint. Embedding vectors live in the `.kv` artifact and are loaded separately. This separation allows:

- Retraining embeddings without touching model weights
- Retraining model layers with frozen embeddings
- Swapping codebooks between models that share the same architecture

---

Grammar

The 5DG grammar used for English sentence parsing is documented in Grammar.md. The parser implementation lives in `bin/parse.py` and loads its CFG rules from `data/grammar.cfg`.

Sigma and Pi Layers

Given a weight matrix ($W \in \mathbb{R}^{m \times n}$) and input vector ($x \in \mathbb{R}^n$), the output vector ($y \in \mathbb{R}^m$) is computed as:

For the Sigma layer:

$$y_j = Wx + b$$

$$y_j = b_j + \sum_{i=1}^{n} W_{ji} x_i$$

For the Pi layer:

$$y_j = b_j \cdot \prod_{i=1}^{n} (1 + W_{ji} x_i)$$

This is not invertible, but if we are allowed twice as many outputs as inputs, we may: do inversion as:

1. Define:
$$s_j := \sum_i \tanh^{-1}(w_{ji} x_i) = Wx$$
2. In forward pass, use:
$$y_j = b_j \cdot \prod_i (1 - \tanh(w_{ji} x_i)), \quad z_j = b_j \cdot \prod_i (1 + \tanh(w_{ji} x_i))$$
3. In reverse pass, compute:
$$\gamma_j = \frac{1}{2} \log\left(\frac{z_j}{y_j}\right) = \sum_i \tanh^{-1}(\tanh(w_{ji} x_i)) = \sum_i w_{ji} x_i \Rightarrow \gamma = Wx$$

4. Finally:

$$x = W^{-1}\gamma$$

Dimensionality Constraints

- The output dimensionality of the input layer must be equal to the output dimensionality of the perceptual layer, since the conceptual layer operates on both.
- The input dimensionality of the symbolic layer must be equal to the input dimensionality of the perceptual layer, since they both operate on the output of the conceptual layer.
- The input dimensionality of the output layer must be equal to the sum of the output dimensionalities of the symbolic layers.

Pi Layer Derivation

$$e^{y_j} = e^{b_j} + \sum_{i=1}^{n} e^{1+W_{ji}x_i}$$

$$\log(e^{y_j}) = \log(e^{b_j} + \sum_{i=1}^{n} e^{1+W_{ji}x_i})$$

$$\log(e^{y_j}) = \log(e^{b_j}) \cdot \log\left(\sum_{i=1}^{n} e^{1+W_{ji}x_i}\right)$$

$$y_j = b_j \cdot \prod_{i=1}^{n} \log(e^{1+W_{ji}x_i})$$

$$y_j = b_j \cdot \prod_{i=1}^{n} (1 + W_{ji}x_i)$$

Now, is there a way to write $e^{1+W_{ji}x_i}$ *as a matrix product,* $W_{ji}x_i$ *?* If we write W and x in the log domain first, we have:

DerivedModel: Ergodic Exploration via Adaptive Bias-Variance Control

BasicModel implements a novel approach to neural network optimization that decouples **what** the network learns (weights W) from **how much it explores** (a scalar α governing the bias-variance tradeoff). Two fundamentally different algorithms operate in tandem:

Component	Algorithm	What it does
Weights W	Standard Adam	Gradient descent on the loss landscape
Tradeoff α	Gradient Energy Sensor	Measures loss-surface curvature to set exploration level

The key insight: α **is not trained by gradient descent.** It is a *sensor* that reads the gradient energy flowing through the temperature parameter and converts it into an exploration policy.

---

1. The Ergodic Weights

In this model, weights are not treated as fixed constants. Each layer instead uses an effective weight matrix sampled from a locally specified ergodic distribution, with the learned tensor and the stochastic tensor defining the current mixture:

$$W_{\text{eff}} = \alpha \cdot W + (1 - \alpha) \cdot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

So the layer never acts on a bare, timeless weight matrix; it acts on the current sample W_{eff}, while W stores the learned structure that increasingly shapes that local distribution as α rises. This perspective is adjacent to the broader literature on stochastic sampling in learning, including Monte Carlo methods, stochastic approximation, and noise-injected views of neural network optimization.

We define two derived quantities from the single scalar α:

$$\text{bias} = \alpha, \qquad \text{temp} = 1 - \alpha$$

This is a **direct encoding of the bias-variance tradeoff**:

- $\alpha = 0$: Pure exploration. $W_{\text{eff}} = \varepsilon$. The network is random noise.
- $\alpha = 1$: Pure exploitation. $W_{\text{eff}} = W$. The network uses only learned weights.
- $0 < \alpha < 1$: Interpolation between learned structure and stochastic exploration.

Training begins at $\alpha = 0$ (full exploration) and the system autonomously transitions toward exploitation as the loss landscape flattens.

---

2. Why Standard Adam Fails on α

The temperature parameter $T = 1 - \alpha$ receives gradients via:

$$\frac{\partial \mathcal{L}}{\partial T} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \varepsilon_{ij}$$

Because ε is **re-sampled every forward pass**, this gradient is **zero-mean**:

$$\mathbb{E}\left[\frac{\partial \mathcal{L}}{\partial T}\right] = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \underbrace{\mathbb{E}[\varepsilon_{ij}]}_{=0} = 0$$

Standard Adam maintains a first moment (running mean) and a second moment (running variance), then computes the update as $m/\sqrt{v}$. For a zero-mean gradient:

$$m_t \approx 0 \quad \Longrightarrow \quad \frac{m_t}{\sqrt{v_t}} \approx \frac{0}{\sqrt{v_t}} = 0$$

Adam produces no update. The first moment kills the signal. This is correct behavior for Adam — it correctly identifies that there is no consistent gradient direction — but it means Adam cannot be used to tune α.

---

3. The Gradient Energy Sensor

3.1 Modified Second-Moment Estimator We discard the first moment entirely and use only the second moment, but as **output** rather than normalizer:

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2$$

where $g_t = \partial \mathcal{L} / \partial T$ is the temperature gradient at step t, and $\beta = 0.999$ is the EMA decay rate.

3.2 Bias Correction Identical to Adam's bias correction. Since $v_0 = 0$, early estimates are biased toward zero:

$$\hat{v}_t = \frac{v_t}{1 - \beta^t}$$

3.3 What $\hat{v}_t$ Measures The second moment of a zero-mean random variable is its variance:

$$\mathbb{E}[g_t^2] = \mathrm{Var}(g_t) = \sum_{i,j} \left(\frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \right)^2 = \left\| \frac{\partial \mathcal{L}}{\partial W_{\text{eff}}} \right\|_F^2$$

This is the **squared Frobenius norm** of the loss gradient with respect to the effective weights — a scalar measure of how much the loss surface cares about weight perturbations. We call this the **gradient energy**.

Gradient energy	Meaning	Desired behavior
High $\hat{v}_t$	Loss is sensitive to weight changes	Keep exploring (low α)
Low $\hat{v}_t$	Loss surface is flat	Exploit learned weights (high α)

3.4 The Alpha Update Rule

$$\alpha_t = \frac{1}{1 + \tau \cdot \sqrt{\hat{v}_t}}$$

where τ (`global_temp`) is a tunable sensitivity knob.

Properties:

- $\sqrt{\hat{v}_t} \to 0 \implies \alpha_t \to 1$ (exploit when gradients vanish)
- $\sqrt{\hat{v}_t} \to \infty \implies \alpha_t \to 0$ (explore when gradients are large)
- $\alpha_t \in (0, 1)$ always — the sigmoid-like shape prevents saturation
- τ controls the transition speed: large τ means more exploration at equivalent gradient energy

After computing α_t, the bias and temperature are derived:

$$\text{bias}_t = \alpha_t, \qquad T_t = 1 - \alpha_t$$

3.5 Layer-Local Certainty The scalar α_t is a **global layer policy**, but individual output neurons do not need to converge at the same rate. Each ergodic layer therefore maintains a certainty vector

$$c_t \in [0, 1]^{n_{\text{out}}}$$

whose entries are tuned from two separate signals kept outside the core ergodic update: a forward-pass activation statistic and the historical gradient energy of the learned per-output parameters in that layer.

The forward signal is a bounded summary of activation magnitude:

$$f_{t,j} = \tanh(\mathbb{E}[|y_{t,j}|])$$

The gradient signal is:

$$v_{t,j}^{(c)} = \beta_c v_{t-1,j}^{(c)} + (1-\beta_c) \cdot g_{t,j}^2$$

$$\hat{v}_{t,j}^{(c)} = \frac{v_{t,j}^{(c)}}{1-\beta_c^t}, \qquad g_{t,j}^{(c)} = \frac{1}{1+\tau_c\sqrt{\hat{v}_{t,j}^{(c)}}}$$

The final local certainty is a blend of the forward and gradient views:

$$c_{t,j} = \lambda_f f_{t,j} + \lambda_g g_{t,j}^{(c)}, \qquad \lambda_f + \lambda_g = 1$$

This yields neuron-wise bias/variance coefficients:

$$\text{bias}_{t,j}^{\text{local}} = c_{t,j} \cdot \text{bias}_t$$

$$T_{t,j}^{\text{local}} = T_t + (1-c_{t,j}) \cdot \text{bias}_t$$

so that $\text{bias}_{t,j}^{\text{local}} + T_{t,j}^{\text{local}} = 1$ whenever dropout is inactive. High-certainty outputs stabilize early and lean on learned weights; low-certainty outputs keep more noise and continue exploring. This lets earlier or easier features settle before later or less reliable ones without changing the global ergodic algorithm.

---

4. Comparison: Adam vs. Gradient Energy Sensor

Property	Adam (for W)	Gradient Energy Sensor (for α)
Type	Optimizer (gradient descent)	Sensor (measurement)
First moment m_t	Yes — tracks gradient direction	No — direction is zero-mean, useless
Second moment v_t	Yes — normalizes step size	Yes — **is the output**
Role of $\sqrt{v_t}$	Denominator (adaptive learning rate)	Numerator (gradient energy estimate)
Update rule	$W \leftarrow W - \eta \cdot m/\sqrt{v}$	$\alpha \leftarrow 1/(1+\tau\sqrt{\hat{v}})$
Descent?	Yes — follows gradient downhill	No — maps energy to a policy
Zero-mean safe?	No — produces $0/\sqrt{v} = 0$	Yes — by design

---

5. Derivation: Why Zero-Mean Implies Drop m_t

Let $g_t = \nabla_T \mathcal{L}$ be the temperature gradient. Since noise ε is i.i.d. each step:

$$g_t = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \varepsilon_{ij}^{(t)}$$

The first moment EMA is:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

Taking expectations:

$$\mathbb{E}[m_t] = \beta_1 \mathbb{E}[m_{t-1}] + (1-\beta_1) \underbrace{\mathbb{E}[g_t]}_{=0} = \beta_1 \mathbb{E}[m_{t-1}]$$

By induction: $\mathbb{E}[m_t] = \beta_1^t \mathbb{E}[m_0] = 0$. The first moment converges exponentially to zero. It carries **no information** about the loss landscape — only noise from finite sampling. Keeping it would add variance to the estimator without adding signal.

The second moment, however, converges to a meaningful quantity:

$$\mathbb{E}[v_t] \to \mathbb{E}[g_t^2] = \left\| \nabla_{W_{\text{eff}}} \mathcal{L} \right\|_F^2$$

This is exactly the gradient energy — the quantity we need.

---

6. Dropout via Bias

The `bias` and `temp` parameters are passed through the full layer stack to support **dropout regularization**:

$$\text{bias}_t^{(\text{drop})} = \begin{cases} 0 & \text{with probability } p \\ \alpha_t & \text{with probability } 1-p \end{cases}$$

When bias is dropped to zero, the layer acts as pure noise regardless of α. This is analogous to standard dropout but operates on the bias-variance tradeoff rather than individual activations. The temperature $T = 1 - \alpha$ remains unchanged so that the noise contribution is consistent.

Seen this way, dropout is just a schedule on bias. Ordinary Bernoulli dropout is the binary case in which the bias is either kept at α_t or forced to 0 for that step, while annealed dropout corresponds to increasing the expected bias over training so the layer spends less time in pure-noise mode and more time exploiting learned structure. More generally, nonzero temperature bakes regularization directly into the model because every forward pass perturbs the effective weights, discouraging brittle co-adaptation and rewarding structure that survives stochastic variation.

Every layer signature accepts `(bias, temp)` to support this:

```
def forward(self, x, bias=1.0, temp=0.0):
    W_eff = bias * W + temp * noise
    ...
```

---

7. The `global_temp` Sensitivity Knob

The parameter τ (`global_temp`) scales how responsive α is to gradient energy:

$$\alpha = \frac{1}{1 + \tau \cdot \sqrt{\hat{v}}}$$

τ	Effect
$\tau = 0$	$\alpha = 1$ always — pure exploitation, no exploration
$\tau \ll 1$	Aggressive exploitation — quickly converges to $\alpha \approx 1$
$\tau = 1$	Balanced — α tracks gradient energy on a natural scale
$\tau \gg 1$	Conservative — maintains high exploration even with moderate gradients

In practice, τ can itself be scheduled (e.g., annealed from high to low) to implement a coarse exploration-to-exploitation curriculum.

---

8. Initialization and Bootstrap

Problem: If we initialized at $\alpha = 1$ (pure exploitation), the noise term vanishes and the temperature gradient $\partial\mathcal{L}/\partial T = 0$. The sensor would read zero gradient energy and keep $\alpha = 1$ forever — an **exploitation trap**.

Solution: Initialize at $\alpha = 0$ (pure exploration):

```
self.alpha       = nn.Parameter(torch.tensor(0.0), requires_grad=False)
self.bias        = nn.Parameter(torch.tensor(0.0), requires_grad=False)
self.temperature = nn.Parameter(torch.tensor(1.0), requires_grad=True)
```

At $\alpha = 0$:

- $W_{\text{eff}} = \varepsilon$ — pure noise carries gradients through the network
- The temperature gradient is nonzero: $g_t = \sum_{ij}(\partial\mathcal{L}/\partial\varepsilon_{ij}) \cdot \varepsilon_{ij} \neq 0$
- The sensor measures initial gradient energy and begins tuning α upward
- As W learns useful structure, gradient energy decreases and α rises naturally

In this architecture, zero-initializing W is not only safe but often cleaner than random initialization. In an ordinary network, zero initialization is bad because it fails to break symmetry, but here symmetry is already broken by the sampled noise ε in W_{eff}. Since training begins at $\alpha = 0$, the learned weights do not contribute to the forward pass anyway, so random weight initialization only injects arbitrary structure into a phase that is meant to be pure exploration. Starting from $W = 0$ makes that exploratory regime honest and lets useful structure enter only when the bias toward learned weights increases.

---

9. Layer Architecture

All layers follow the effective-weight pattern with `(bias, temp)` signatures:

LinearLayer

$$y = x \cdot (\text{bias} \cdot W + \text{temp} \cdot \varepsilon_W) + \text{bias} \cdot b + \text{temp} \cdot \varepsilon_b$$

ReversibleLinearLayer (SVD decomposition)

$$W = U\Sigma V^\top$$

Each factor applies the tradeoff independently:

$$y = V \cdot \Sigma \cdot U^\top \cdot x$$

where each rotation and scaling layer uses `(bias, temp)` internally.

ReversibleRotationLayer Parameterized by Givens angles θ_k:

$$\theta_k^{\text{eff}} = \text{bias} \cdot \theta_k + \text{temp} \cdot \varepsilon_k$$

ReversibleDiagonalLayer Parameterized by log-singular-values λ_k:

$$\lambda_k^{\text{eff}} = \text{bias} \cdot \lambda_k + \text{temp} \cdot \varepsilon_k$$

---

10. Training Loop Integration

```
# 1. Standard Adam optimizer for W (excludes temperature)
optimizer = torch.optim.Adam(model.getParameters(), lr=lr)

# 2. Forward pass computes W_eff = bias*W + temp*noise
loss = model(x, y)

# 3. Backward pass: Adam gets dL/dW, temperature gets dL/dT
loss.backward()

# 4. Adam updates W
optimizer.step()

# 5. The sensor updates alpha, while local certainty blends
#    forward activity and gradient history
model.paramUpdate()  # calls alpha_update() on each ErgodicLayer
```

Note that `getParameters()` **excludes** the temperature parameter:

```
def getParameters(self):
    params = [p for n, p in self.named_parameters() if n != "temperature"]
    return params
```

Temperature is excluded from Adam because Adam would produce zero updates on it (see Section 2). Instead, the gradient energy sensor reads `temperature.grad` directly.

---

11. Summary of the Full Algorithm

Initialize:

$$\alpha_0 = 0, \quad v_0 = 0, \quad t = 0, \quad \beta = 0.999$$

Each training step:

1. Sample $\varepsilon \sim \mathcal{N}(0, I)$
2. Compute $W_{\text{eff}} = \alpha_t W + (1 - \alpha_t)\varepsilon$
3. Forward pass: $\hat{y} = f(x; W_{\text{eff}})$
4. Compute loss $\mathcal{L}(\hat{y}, y)$
5. Backward pass: compute $\nabla_W \mathcal{L}$ and $g_t = \nabla_T \mathcal{L}$
6. **Adam** updates W: $W \leftarrow W - \eta \cdot \hat{m}_t / \sqrt{\hat{v}_t^{(W)}}$
7. **Sensor** updates α, and a separate certainty routine blends forward activity with gradient history:

$$v_t \leftarrow \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2$$

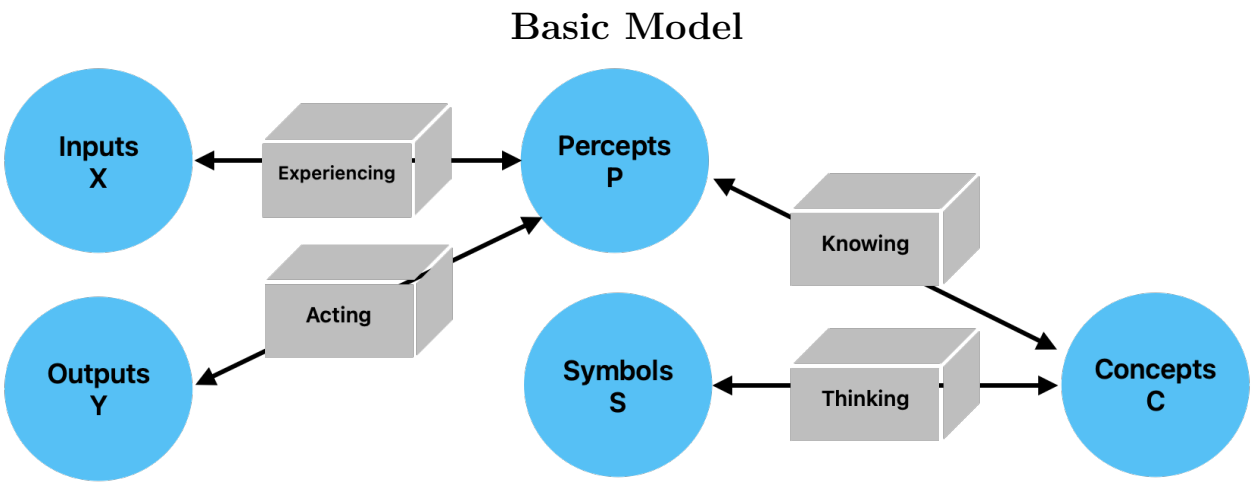
$$\hat{v}_t = \frac{v_t}{1-\beta^t}$$

$$\alpha_{t+1} = \frac{1}{1+\tau\sqrt{\hat{v}_t}}$$

$$c_{t+1,j} = \frac{1}{1+\tau_c\sqrt{\hat{v}_{t,j}^{(c)}}}$$

$$\text{bias}_{t+1,j}^{\text{local}} = c_{t+1,j} \cdot \text{bias}_{t+1}, \qquad T_{t+1,j}^{\text{local}} = T_{t+1} + (1 - c_{t+1,j}) \cdot \text{bias}_{t+1}$$

8. Zero temperature gradient: $g_t \leftarrow 0$



The Basic Model of Cognition is a neural network that consists of four spaces: real space, perceptual space, conceptual space, and symbolic space.

Model Entities:

Entity	Meaning
x, $\hat{x}$, X	Actual and predicted input (experiences), input space
y, $\hat{y}$, Y	Actual and predicted output (actions), output space
p, P	Percepts, perceptual space
c, C	Concepts, conceptual space
s, S	Symbols, symbolic space
W_e	Experiencing, experiential weights
W_a	Acting, action weights
W_k	Knowing, knowledge weights
W_t	Thinking, thought weights

Background

Much of this work is based on the Basic Model of Cognition described briefly at cognitivesciencesociety.org/framework-cognitive More details are provided in the book The Whole Part.

Inputs and Outputs

Inputs and outputs derive from a real space that is ineffable. This essay merely characterizes them relative to other things.

The adaptable parameters of a mind are initially zero, a state known as *tabula rosa*. Learning is initiated by random perturbations of a map relative to its territory, and the degree of that exploration is referred to as *temperature*.

Input are normalized and lie on a hypersphere. They are nonnegative because negative entities do not exist. To paraphrase Saint Augustine, *negative is the privation of the positive*.

Outputs and Inputs are dehumanizing terms: the world is a living world, so it helps to call outputs "actions" and inputs "experiences". When minds perceive and act within the world, their mistakes are called expectation errors and performance errors.

Action is a function of perceptual space and our expectation, where that space is often filtered by conceptual projections.

Performance errors are defined relative to action, desired effect, and exhaustion.

Expectation errors are defined relative to perception and perception-as-reconstructed.

To accommodate an input context window which changes, the input is augmented with relative spatiotemporal encoding by extending its dimensionality. While much of the network can treat these as simply additional features of the input, the attention mechanism must deal with them explicitly since attention is directed within space and time (i.e. it treats the what and where pathways separately).

Perceptual Space

Perceptual spaces are composed of perceptual prototype vectors, or ***percepts***.[1]

Perceptual spaces map onto reality in a straightforward way, at least in so far as we perceive correctly They are bidirectional, although not necessarily symmetric. Technically, they are real similarity spaces with arbitrarily high dimensionality, where logical negation is not defined.

Percepts that do not correspond to objects are *hallucinations*.

Percepts generalize a perceptual space by forming a Voronoi tessellation using prototype and/or exemplar vectors. They are related to one another in virtue of a connection which define a Self Organizing Map. Percepts operate in parallel. Insofar as categories are concerned, they form an embodiment of prototype theory.

The perceptual error function is defined by the mismatch between perception and existing percepts, where that mismatch may exert an influence on the world.

Perceptual accuracy is a measure of the distance between inputs and prototype vectors. That distance is similar to cosine similarity, although to the extent we don't know a perceptual well, it is a simple distance measure in the range (0-1).

Attraction and aversion within a perceptual space manifest as weighted prototype vectors: attraction is a pole at the percepts's location, and aversion is a zero at that location. As a result, they warp perceptual space, making us more or less likely to perceive things that we love or hate.

Perfectly recognized percepts lie on the unit hypersphere within conceptual space.

Two percepts can be joined into one concept without loss of characterization.

Conceptual Space

Concepts create boundaries in conceptual space, a space which is grounded in the support vectors of perception. By creating boundaries, they support logical negation. Due to this relativity, they may refer to negative entities which may or may not exist on perceptual or objective levels.

Conceptual spaces are capable of representing negative entities.

[1] Perceptual space is known as "embedding space" in the current literature on transformer neural net architectures.

Concepts are references that do not exist in the space in which the percepts are defined, but in the space formed by the collections of perceptual activations. Thus, the space of concepts can not be directly overlaid on the space of percepts, because it occurs at a different epistemic level, although conceptual space can be projected onto perceptual space.

Concepts that do not correspond to percepts are *false*.

Concepts collect percepts into fuzzy sets. Percepts are weighted by a degree of membership, which allow concepts to perform several logical operations such as and, or, and not. They can be seen either as linear separating hyperplanes which partition conceptual space into two parts, or as sets which are defined as a linear combination of the percept activations.

Concepts are relative: they are meaningful only insofar as they create a partition of a meaningful space.

As the concepts of a mind approach certainty, the slope of the decision boundary becomes infinite.

There is a single state of unknowing which is common to all knowledge vectors, which does not categorize space because the slope of the activation function is zero.

It is also the case that knowledge vectors of length zero are undirected, which seems like a state of unknowing which is slightly different than slightly different the measure of certainty. Informally, the concept must be learned before its certainty can be assessed.

Thus, conceptual spaces are similarity spaces that are partitioned by decision boundaries that impose positivity and negativity on a hypersphere of knowing.

Percepts are incorporated into meronomies through the use of concepts, which create transitive part hierarchies.

When symbols are so incorporated, the referential nature of symbols creates intransitive part hierarchies.

Within conceptual space, attraction and aversion to concepts can be expressed as a hyperplane bias (i.e. the tendency to partition reality in a biased way).

Within conceptual space, certainty is expressed as the slope of the activation function. Is it tuned after the other parameters have ceased to change as a result of correct categorization.

Certainty (or clarity) can also be grounded in symbolic space, where symbolic support vectors are taken as *a priori* truth (or at least learned with some certainty reflected in their nonzero magnitude).

Concepts can be split into two precepts without loss of characterization.

Symbolic Space

Symbolic Spaces are composed of percepts that are references to concepts. They are therefore a subtype of perceptual spaces that map onto conceptual spaces.

Because symbols do not have any spatial context, they exist as zero-dimensional points within perceptual space. If we dont have feeling receptors in the brain, there is no positional encoding for this layer.

In virtue of that representation, symbols can be conceptualized and collected into sets. The creation of sets of symbols, since they have no inherent ordering, creates equivalence classes.

In humans, perceptual space is projected onto a 3D subspace within the cerebellum before symbolic encoding.

Symbols are independent, permanent, and integral (at least when interpreted *as* references, even though they ultimately exist within a dependent, temporary, and continuous space).

The top-down, serial activation of a symbolic space differs from the bottom-up activation of perceptual spaces because it casts shadows.

The formation of symbols increases the dimensionality of conceptual space (as would an outer product).

The formation of concepts from percepts decreases the dimensionality of perceptual space (as would an inner product).

Symbols are zero-dimensional entities, or epistemological points. As references, symbols form a discrete perceptual space and can be represented as a binary array.

The participation of symbols in multiple sets creates structural relations which takes the place of positional information.[2]

Symbols are erroneous if their associated concept is not true.

Reasoning

Reasoning within symbolic spaces involves computing with parts and wholes. This may be seen as using Venn diagrams to perform computations on a space, which results in ability to perform Bayesian-type statistics dynamically.

Those probabilities, once learned, can be stored as unidirectional connection weights between concepts, perhaps in conjunction with a part-whole structure imposed on those concepts by their corresponding symbols.

Symmetric Perception: the Single Layer Linear Case

- A symmetric perception network predicts both input and output. By taking advantage of functions that may be surjective in two directions, it is able to compute a bijective mapping.
- The term "*symmetric perception*" refers to the fact that perception is bidirectional: it is an interaction in which we see the world and in which the world sees us. The later fact is often forgotten from an egocentric point of view; culturally, as Edward has observed, perception is seen as a passive process. In the context of neural networks, which often serve as models of the mind, this egocentric view continues: functions are computed that map inputs onto outputs, and the influence of the outputs on the inputs is forgotten. This essay explores the technical aspects of symmetric perception as well as several guiding principles for humanistic mental models.

Symmetric Perception, in the general case, finds a function f(x) which minimizes the MSE with respect to the following:

$$\begin{pmatrix} in \\ out \end{pmatrix} \cdot \begin{pmatrix} f(x) & 0 \\ 0 & f^{-1}(x) \end{pmatrix} = \begin{pmatrix} out \\ in \end{pmatrix}$$

In the linear case, $f(x)$ becomes a single invertible matrix A. In general, the mapping from input to output may not be linear, and the inverse may not exist, in which case the previous set of input/output equations is written as:

$$\begin{pmatrix} in \\ out \end{pmatrix} \cdot \begin{pmatrix} f(x) & 0 \\ 0 & g(x) \end{pmatrix} = \begin{pmatrix} out \\ in \end{pmatrix}$$

This problem bears some resemblance to the standard regression problem $Ax = b$, except that x is known and we wish to find the set of points A (or weights) that map one linear space to another. So the familiar Linear Least-Squares solution to the equation $x = \left(A^\top A\right)^{-1} A^\top b$ becomes an attempt to find a matrix A such that:

$Ax = b$ and $A^{-1}b = x$

This solution is seen as the "best" solution by neural networks that wish to determine a weight matrix A that maps from x onto b. That much is shared between the two problems, although there are nonlinearities in the general neural network case which enable a much better fit. This solution is optimal in the sense that it minimizes the Sum of Squared Errors, where the errors are defined as the residuals when b is projected into the column space of A.

[2]For example, {0,1} and {0,1}, if those sets are orthogonal to one another, pose no constraint on the location of the 1 within the 2x2 grid that they partition. However, {0,1,1} and {0,1} partition a 3x2 space and simultaneously impose the constraint that two 1's must lie on the same row. This is a fact which is not present in either the row set or the column set.

However, the principle of retrocausality suggests an additional objective; namely, we wish to create a matrix such that it is optimal both for the the transformation of the input to the output and the output to the input.

In general, we want a matrix A such that minimizes reconstruction loss over both $Ax = b$ and $A^{-1}b = x$. Since we are simultaneously minimizing the error vectors of each, and since the column space of a matrix is identical to the column space of its inverse, we will end up with a matrix A that contains the vectors x and b within both its column space and row space (which is necessary because we are seeking a bijective function).

As a first approximation, we might write A as a product of two matrices, each of which performs a perfect surjection: $A \cdot A^{-1} = I$

$$(\frac{b}{2} * x^{-1}) \cdot (\frac{x}{2} * b^{-1}) = I$$

However, this will still be only an approximate solution, since it is unlikely that a somewhat arbitrary combination of both matrices will result in the correct orthogonal matrix. To find the best matrix A involves either an iterative solution as would be performed by NN learning, ping-pong style, or (theoretically?) via multiple round trips consisting of Gram-Schmidt orthogonalization with respect to the matrices A and A^{-1}.

Symmetric Perception: the Single Layer Nonlinear Case

If $f(x)$ and $g(x)$ share a common weight matrix W, we may write the equations for symmetric perception as:

Forward computation and gradient:

$$\hat{y} = f(g^{-1}(x)W)$$

$$e_y = \frac{1}{2}(\hat{y} - y)^2$$

$$\frac{\partial e_y}{\partial \hat{y}} = \hat{y} - y$$

$$\frac{\partial \hat{y}}{\partial W} = \delta_f(W^\top \cdot g^{-1}(x))$$

$$\frac{\partial W}{\partial x} = \delta_g(x)$$

$$\Delta W_y = \eta \frac{\partial e_y}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial W} g^{-1}(x)$$

$$\Delta W_y = \eta(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x)$$

Reverse computation and gradient:

$$\hat{x} = g(W^\top f^{-1}(y))$$

$$e_x = \frac{1}{2}(\hat{x} - x)^2$$

$$\frac{\partial e_x}{\partial \hat{x}} = \hat{x} - x$$

$$\frac{\partial \hat{x}}{\partial W} = \delta_g(W \cdot f^{-1}(y))$$

$$\frac{\partial W}{\partial y} = \delta_f(y)$$

$$\Delta W_x = \eta \frac{\partial e_x}{\partial \hat{x}} \frac{\partial \hat{x}}{\partial W} f^{-1}(y)$$

$$\Delta W_x = \eta(\hat{x} - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)$$

Since the single matrix W is shared, we might combine the partial derivatives of the input and output errors via sum and set the gradient to zero to explore the solution space, which seems to minimize the cross-covariance between the input and output spaces:

$$\frac{\partial e_y}{\partial W} + \frac{\partial e_x}{\partial W} = \eta(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x) + \eta(\hat{x} - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)$$

If we assume that the gradients never go to zero, then we have the sum of the partial derivatives with respect to the error going to zero when both $x = i$ and $y = o$. However, it may be the case that $xy - iy = ox - yx$, which means that the simultaneous gradient descent is ping-ponging between the two cost functions. So a better cost function would be a multiplication of the two error partials, which is an intersection of the space that satisfies both equations. In that case, we can multiply the terms to get a better understanding of the error surface that we are minimizing with respect to the input and the output:

$$\frac{\partial e_y}{\partial W} \frac{\partial e_x}{\partial W} = \eta(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x)(\hat{x} - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)$$

Distance Metrics

$$d(x_1, x_2) = \frac{\|x_1\|^n \cdot \|x_2\|^n}{\|x_1 - x_2\|^{2n}}$$

Which can be written in the Z-plane as:

$$d(x_1, x_2) = \|x_1 - x_2\| \cdot \frac{(x - z_1)}{(x - z_2)}$$

Where z_1 is a zero and z_2 is a pole.

Exploration/Exploitation as a function of temperature:

$$y = x \cdot \frac{(W + rt)}{\|W\|}$$

Where:

- $\boldsymbol{x}$, $\boldsymbol{y}$ are the input and output vectors
- $\boldsymbol{W}$ is the weight matrix

- $\boldsymbol{r}$ is a random vector
- $\boldsymbol{t}$ represents temperature

Transfer Functions

By using rotation matrices with a tunable parameter Θ before and after the projection from one space to the other, and by restricting the projection matrix to be diagonal, it is possible to restrict the solution space to an eigendecomposition of the orthonormal basis that maps input to output (i.e. it calculates the SVD of the matrix). For the 2D case, this can be written:

$$y = \begin{bmatrix} \cos\theta_1 & -\sin\theta_1 \\ \sin\theta_1 & \cos\theta_1 \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0 \\ 0 & \Sigma_2 \end{bmatrix} \begin{bmatrix} \cos\theta_2 & -\sin\theta_2 \\ \sin\theta_2 & \cos\theta_2 \end{bmatrix} x$$

It may also be possible to use a nontunable scalar function as the transfer function and not restricting the matrix to be diagonal, which is approximately how neural networks apply nonlinearities to the sum of products by weight matrices.

$$f(x) = \sin(x)$$

$$g(x) = \cos(x)$$

$$f^{-1}(x) = \arcsin(x)$$

$$g^{-1}(x) = \arccos(x)$$

$$\partial_f(x) = \cos(x)$$

$$\partial_g(x) = \sin(x)$$

$$\partial_{f^{-1}}(x) = \frac{1}{\cos(\arcsin(x))}$$

$$\partial_{g^{-1}}(x) = \frac{1}{\sin(\arccos(x))}$$

If we write the forward equations and gradients of $f()$ and $g()$ using these functions, we end up with the following formulation:

$$\hat{y} = \sin(\cos^{-1}(x)W)$$

$$\hat{x} = \cos(W^\top \sin^{-1}(y))$$

$$\Delta W_y = \eta(\hat{y} - y)\cos(W^\top \cdot \cos^{-1}(x))\cos^{-1}(x)$$

$$\Delta W_x = \eta(\hat{x} - x)\sin(W \cdot \sin^{-1}(y))\sin^{-1}(y)$$

The important thing to notice is that the reconstruction of the reverse perception and the calculation of the gradient of forward perception involve common terms, and may be simplified:

$$\hat{y} = f(g^{-1}(x)W)$$

$$\hat{x} = g(W^\top f^{-1}(y))$$

$$\Delta W_y = \eta(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x)$$

$$\Delta W_x = \eta(\hat{x} - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)$$

$$\Delta W_y = \eta(f(g^{-1}(x)W) - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x)$$

$$\Delta W_x = \eta(g(W^\top f^{-1}(y)) - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)$$

$$\Delta W_y \Delta W_x = \eta(f(g^{-1}(x)W) - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x) \cdot (g(W^\top f^{-1}(y)) - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)$$

If we take the product of both:

$$\Delta W_y \Delta W_x = \eta(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x) \cdot (\hat{x} - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)$$

$$\Delta W_y \Delta W_x = \eta(\hat{x} - x)(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))\delta_g(W \cdot f^{-1}(y))f^{-1}(y)g^{-1}(x)$$

Alec Rogers

September 24, 2025

Abstract

Problem

Society has become highly dependent on machine minds, but we do not have a good theory of those minds. Although human and machine minds have considerable functional overlap, their design goals are considerably different, and treating a machine mind as a human mind can have significant negative consequences. Therefore, we should have a different "theory of mind" when we interact with machine minds, and our theory should be informed by how those minds are designed and trained.

Background

To develop this theory of mind, it is helpful to anthropomorphize machines because we naturally understand them in relation to human minds. Therefore, this essay focusses on three main questions: what are machine minds, what do machine minds feel, and what do machine minds know.

While these changes do not allow us to adopt a human theory of mind, they do permit of a theory of mind that is significantly less broken and inhumane.

Solution

As a result of this analysis, we also propose three specific measures to make a machine mind more humanistic or natural: weight ergodicity, network invertibility, and output certainty. Weight ergodicity refers to the fact

that weights are samples from a random (ergodic) process, rather than fixed constants. Network invertibility refers to the fact that the network architecture is bidirectional and partially invertible. Output certainty refers to the fact that a network should know that it does not know, as opposed to merely knowing the probability of one answer as opposed to another. To illustrate these principles, the performance of several models implementing these paradigms is compared to a traditional model. All models are configured with a 20-neuron, single hidden layer and tested on the MNIST dataset.

Network invertibility is important with respect to developing a meaningful semantic embedding: the symbols of an LLM should correspond to something and we should be able to know to what they correspond, as opposed to being merely abstract tokens inside of Searl's Chinese translation engine.

Background

The scope of this paper is Large Language Models, or reinforcement learning where the inputs and outputs are known and the weights of the network are unknown (i.e. the transformer architecture).

Weight Ergodicity

Measurement theory, when applied to a neural network, views the weights as samples from a random (ergodic) distribution. According to that insight, the network parameters and especially the weight matrix W is represented as follows:

$$W = \mu M + \sigma V$$

where μ and σ are the bias and variance parameters that determine the contribution of the random process. The bias and variance parameterize a noisy signal where M has a norm of 1 and variance of zero and an unbiased variance V with unit variance. Note that the matrix M is analogous to the weight matrix in a typical neural network while V is sampled from a standard normal distribution.

For simplicity, the mean and variance of all weights are characterized here by a single *temperature* parameter for each layer that is updated via the ADAM algorithm. While it may be desirable to have a temperature associated with each input weight, this is not necessary to compare to a normal network in order to measure performance.

Another simplification in this context is to use the temperature parameter to determine both the basis and the variance. Given the temperature *temp*, the μ and σ parameters are calculated as follows:

$$\mu = 1 - temp$$

$$\sigma = temp$$

In this experiment, the temperature of the ergodic weight process begins at 1 and decreases to 0 using the ADAM algorithm.

There are three significant advantages to using ergodic weights. The first advantage is that a learning rate is not necessary, since the biases have different contributions as a result of the randomness and converge as the temperature decreases. Thus, the adaptive learning rate schedule is reinterpreted as a process of simulated annealing. The second advantage is that the weights can be initialized to zero, since they find different locations in weight space over time. It is hypothesized that this process of annealing will help the model to bounce out of local minima and saddle points within its weight space more often than taking a step in the direction opposite to the gradient, which is often prone to deterministic and oscillatory behavior. The third advantage is described in the literature on simulated annealing, according to which there is a theoretical guarantee of convergence to the global optimum if run for an infinite number of iterations.

Note that the standard *dropout* algorithm may be interpreted as a per-neuron variance in the bias parameter. Thus, while dropout explicitly sets neurons to zero with some frequency, the same type of regularization will occur naturally as a result of the difference over time of the weight biases.

Network Invertibility

The egocentric point of view tends to see perception as a passive process, while physics suggests that it is bidirectional. That bidirectionality within a neural network can be approximated via the invertibility of its weight multiplications and activation functions. Even when the network is not invertible, bidirectionality provides an important role in backpropagation and network topologies such as autoencoders that perform a decoding step that is analogous to the reverse of the encoding step. In modern practice, the topological similarity of encoders and decoders does not typically extend to shared weights. Here, we briefly examine the possibility of simultaneously training the network in both the forward and reverse direction: the forward encoding step in which the outputs are predicted from the input, and the reverse decoding step in which the inputs are predicted from the (possibly predicted) outputs. For a related experiment in sharing the weight matrix between controller (policy) and critic (value) networks, see *Rogers et al 2001*.

In this experiment, the output is a one-hot encoded vector corresponding to the predicted MNIST digit, a classification task that is clearly not a bijective (invertible) function. To address this situation, the inputs are predicted from the outputs of the *penultimate* network layer; thus, the network is composed of a bijective front and a surjective back. We characterize these components respectively as the *recognizer* (or representer) and the *generalizer*.

Output Certainty

The error function of a neural network corresponds to its desire. Modern machine minds use cross-entropy loss, so their output distribution *must* classify the output. This does not allow the network to express the certainty that the input belongs to a given class or not; rather, it expresses the probability that the input belongs to one class *as opposed to another*. To remedy this situation, a variation of cross-entropy loss is used which blends standard cross-entropy loss with a product of cross-entropy loss and the norm of the prediction. An output value of zero is penalized by cross-entropy loss, but any incorrect prediction carries an additional penalty in proportion to its magnitude to penalize overconfidence. The formula for the certainty-weighted cross-entropy version of the loss function is:

$$error = -\sum_{c=1}^{N} t_c \log(p_c)$$

$$certainty = |\widehat{y}|$$

$$surprise = (\alpha) \cdot certainty \cdot error + (1 - \alpha) \cdot error$$

cosine similarity

Amplitude certainty

$$error = (\widehat{y} - y)^2$$

Methods

The software used to run these experiments is a combination of Python and PyTorch (the Python code is provided in the appendices).

The data is the MNIST dataset, chosen because it is simple and well-known. It consists of 60,000 training images and 10,000 test images, each 28x28 pixels in size, with a monochromatic bit depth of 256. Their

target labels are provided as output. The data is preprocessed by removing its mean and variance, and is shuffled between trials.

For all paradigms, the input is a 28x28 element array and the output is a one-hot encoded representation of the target class. The architecture for all paradigms is a 20 hidden unit neural network. The batch size is 10, and each network is run for 9 epochs. To generate confidence intervals for model accuracy, each paradigm is run for 7 trials to generate error bars reflecting the standard deviation of the accuracy values.

The standard architecture that serves as a benchmark for the other paradigms uses a RelU nonlinearity, and a learning rate of 0.01 that is updated via the ADAM method. Cross-entropy loss is used for the error function.

The base ergodic model uses weights initialized to zero and a single per-layer temperature parameter that governs the bias/variance trade-off of the ergodic weights. The temperature is updated with the ADAM method. Dropout is used for the middle layer, which means that both the bias and variance of the ergodic weights are set to zero with a frequency of 0.75.

Five models were evaluated: the base model and four models using the ergodic weight updating scheme. Of the models with ergodic weights, one model has simply ergodic weights, one has input normalization, one reversible model uses separate forward and backward layers, and one reversible model uses a single invertible weight matrix. For the invertible model, the inversion of the weight matrix is typically computationally expensive. Therefore, the weight matrix is represented in SVD form, as a rotation matrix followed by an eigenvalue matrix followed by another rotation matrix. In virtue of this sparsely-parameterized decomposition, the rotation matrices can be computed by Givens rotations, and the eigenvalue matrix stores only diagonal elements. Therefore, direct matrix inversion and its computational cost are avoided. That said, doing so requires a more computationally expensive forward pass and a somewhat circuitous back propagation step.

The weight update equations for the ergodic model do not use a learning rate. So compared to traditional gradient descent equations that look as follows for a given learning rate parameter α:

$$err = (y - \widehat{y})^2$$

$$\triangle_W = \alpha(y - \widehat{y})\nabla_F x$$

$$err = (y - \widehat{y})^2$$

$$e = y - \widehat{y}$$

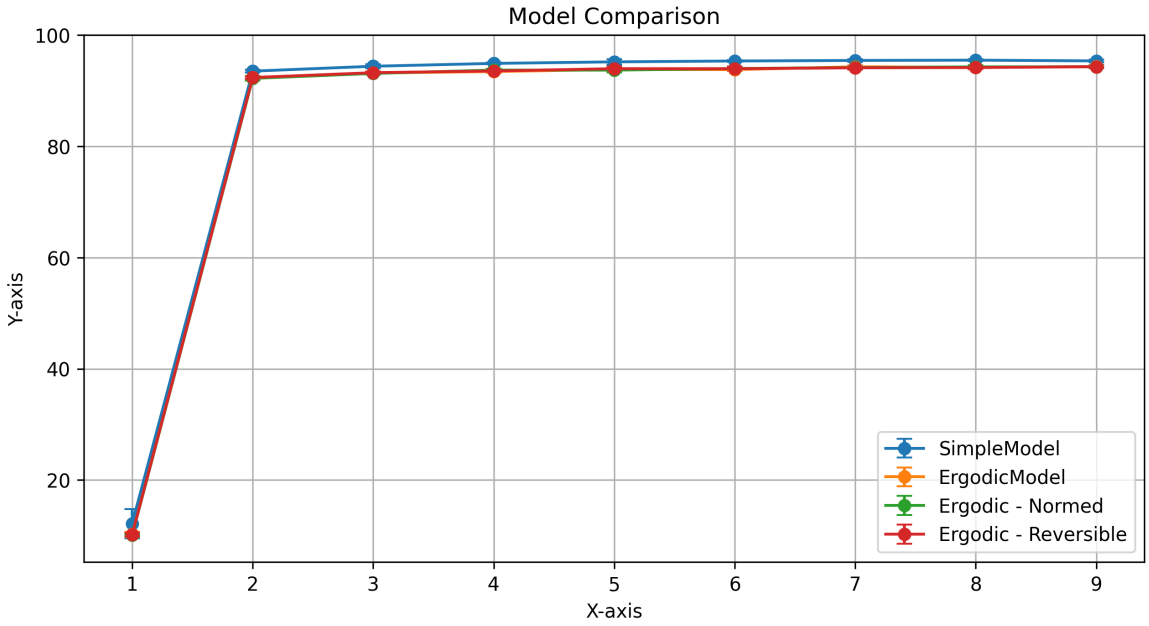
$$c = \alpha\widehat{y}^2 + (1 - \alpha)$$

$$\triangle_W = (ce - \alpha\widehat{y}e^2)x$$

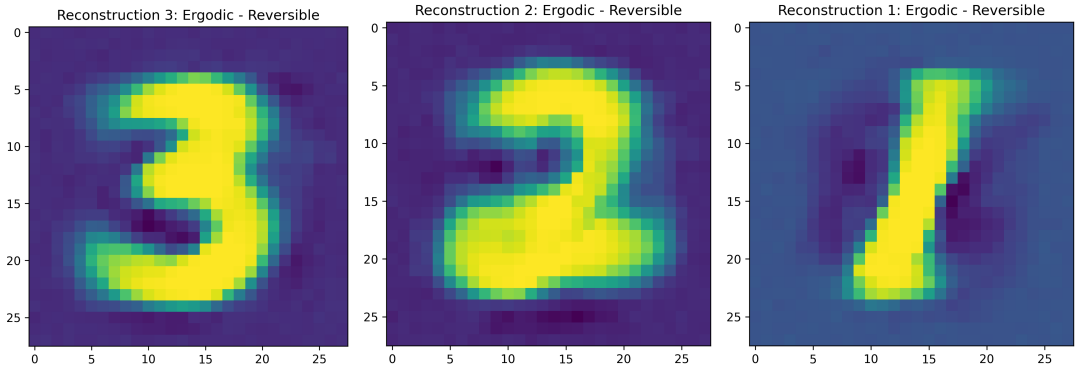
This equation demonstrates that the network is simultaneously trying to minimize error and "incorrect certainty".

Results

The most important result of this experiment is the model comparison with error bars, that demonstrates that the simple model performs slightly better than all proposed paradigms, and that all of the proposed paradigms perform a lmost identically:



Finally, although there is no invertible network to compare it to, we show the reconstruction of the input by running the network in reverse, which uses a separate weight matrix and MSE loss for input reconstruction:



Discussion

The benefits of the novel paradigms described in this paper are as follows:

Random weight initialization is unnecessary, which makes performance less subject to arbitrary initialization. This is visually demonstrated by the error bars in the Model Comparison figure, which show that only the simple model (with random weight variation) has significant variation in performance.

The learning rate is unnecessary, since its role is replaced with weight noise variance, just as the role of dropout is replaced with weight bias variance (which in this experiment is just the regular dropout schedule). Since dropout is folded into the weight adaptation schedule, an integrated algorithm that controls both weight bias and weight variance (and which uses separate parameters for each weight) is suggested for future experiments that follow this paradigm.

The error function encapsulates a measure of certainty in the output in addition to a measure of certainty

between the output choices.

However, there were several experiments that my hardware was not fast enough to evaluate and a number of implementation issues that I did not have the time to address:

The Pearson correlation between the certainty and the accuracy for each digit is insignificant, which suggests that the error function did *not* induce the network to be less certain when it was less accurate. This probably points to a problem in the implementation of the error function: instead of using $certainty = |\hat{y}|$ to reduce the penalty for lack of certainty, we used $certainty = p_c$. This leads to prediction norms that are much higher than 1.0 (since softmax creates a unit-sum PDF at the output layer, while using the norm of the output vector to indicate certainty imposes a contradictory constraint). This might explain the lack of correlation between the certainty and accuracy, which theoretically should correlate quite strongly. Performing the experiment with corrected code (as by using a norm-weighted MSE with only one zero) should yield better correlation between certainty and accuracy for each target class.

Finally, **the use of the autograd algorithm within PyTorch led to weights being tuned directly**, which interferes with the simultaneous tuning of the temperature parameter in a way that is not explicitly captured in this paper. So while the addition of weight variance does make random weight initialization unnecessary, the intricacies of the interaction between the weight tuning and the temperature tuning is hidden behind the autograd mechanism.

Conclusion

Overall, I am optimistic about the strength of the philosophical grounding behind several of the paradigms introduced in this paper. However, due to several implementation issues, I do not feel that the hypotheses were well-tested. Further, even if they had been, the effectiveness of these neural network paradigms in both multilayer neural networks and with more challenging tasks is not clear.

References

Rogers, Shannon, Lendaris; 2001: *A comparison of DHP based antecedent parameter tuning strategies for fuzzy control*, Proceedings of the Joint 9th IFSA World Congress and 20th NAFIPS International Conference, IEEE

Rogers, A; 2003: *Analysis of the Instantaneous Estimate of Autocorrelation*, unpublished

XML Configuration Parameters

Model configuration is specified in XML files (e.g. `data/XOR_exact.xml`). Default values are loaded from `data/model.xml` and overridden by model-specific configs. The schema is defined in `data/model.xsd`.

`<architecture>`

Core model settings. Training and data parameters are in nested sub-elements `<training>` and `<data>` (see below).

Parameter	Type	Default	Description
`modelType`	string	`"simple"`	Model type: `"simple"` (feedforward), `"lm"` (language model with sequence proc
`ergodic`	bool	`false`	Enable ergodic exploration mode. Layers use `W_eff = bias * W + temp * noi`
`certainty`	bool	`false`	Enable per-neuron certainty tracking in ergodic layers. Allows individual neuron
`reshape`	bool	`false`	Reshape input data for sequence processing. Required for `modelType=lm`.
`conceptualOrder`	int	`1`	Order of conceptual processing. Controls how many higher-order conceptual tra
`symbolicOrder`	int	`1`	Order of symbolic processing. Controls how many higher-order symbolic transfo
`processSymbols`	bool	`false`	Enable additional symbolic processing steps.

Parameter	Type	Default	Description
`maskedPrediction`	string	`"NONE"`	Masked prediction mode: `NONE`, `MLM`, `ARLM`, `ARUS`, `RARLM`. See Training.md.
`reconstruct`	string	`"NONE"`	Controls the reverse (reconstruction) pass. `"NONE"` disables reconstruction entir
`maxResponseLength`	int	`64`	Maximum number of characters/tokens to generate during inference. Measured

---

`<architecture><data>`

Data loading and filtering settings.

Parameter	Type	Default	Description
`dataset`	string	`"xor"`	Dataset to load. Determines input/output data via `TheData.load()`. Common va
`minFrequency`	float	`0.0`	Minimum word frequency ratio for vocabulary admission. Words below this thresh
`shardDir`	string	—	Directory containing text shards for streaming datasets (e.g. `"data/fineweb"`).
`numShards`	int	`1`	Number of shards to load from `shardDir`.
`maxDocs`	int	`10000`	Maximum number of documents to load per shard.
`classificationMin`	float	—	Minimum threshold for classification accuracy reporting.
`classificationMax`	float	—	Maximum threshold for classification accuracy reporting.

`<architecture><training>`

Training loop and I/O settings.

Parameter	Type	Default	Description
`numTrials`	int	`1`	Number of independent training runs.
`numEpochs`	int	`3`	Number of training epochs per trial.
`batchSize`	int	`10`	Mini-batch size for training.
`learningRate`	float	`0.001`	Learning rate for the Adam optimizer.
`reconRatio`	float	`0.5`	Weight of reconstruction loss in combined loss: `total = (`
`trainEmbedding`	string	`"NONE"`	Embedding update mode. Controls both the embedding tr
`trainEmbeddingRatio`	float	`0.1`	Weight λ of embedding loss in JOINT mode: $\mathcal{L} = \mathcal{L}_{\text{model}}$
`weightsPath`	string	`"output/BasicModel.ckpt"`	File path for saving/loading model weights checkpoint. Fil
`embeddingPath`	string	—	File path for the word vector store (`.kv` extension, gensim
`autoload`	bool	`true`	Automatically load weights from `weightsPath` on model c
`autosave`	bool	`false`	Automatically save weights after training completes.
`negSamples`	int	`64`	Number of negative samples per positive example for CBO

`<trainEmbedding>` — Embedding Update Modes

Value	Embedding method	Model layers	Gradients through codebook	Description
`NONE`	Frozen	Frozen	No	Inference only; no parameters updated
`CBOW`	CBOW (padded context)	Frozen	No	True CBOW: predict each word from lea
`SBOW`	SBOW (centroid)	Frozen	No	Faster variant: predict each word from le
`ARLM`	Frozen	Backprop	No	Train model layers only; codebook is fixe
`BOTH`	SBOW post-batch	Backprop	Yes	Two optimizers: SBOW updates embedd
`JOINT`	Single backward	Backprop	Yes	Single optimizer: $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{model}} + \lambda \cdot \mathcal{L}$

<trainEmbeddingRatio> — Embedding Loss Weight (JOINT mode)

Parameter	Type	Default	Description
`trainEmbeddingRatio`	float	`0.1`	Weight λ of the embedding co-occurrence loss in JOINT mode. Higher values bi

---

<InputSpace>

The entry point that lifts raw data into the model's internal representation.

Parameter	Type	Default	Description
`nActive`	int	*required*	Sequence length: maximum number of tokens per input. For XOR: 2 (two binary inputs
`nDim`	int	`1`	Dimensionality of each input vector. Set on TheObjectEncoding via XML; not passed to
`nVectors`	int	= `nActive`	Codebook size (total vectors in the space). Defaults to `nActive` for InputSpace.
`nWhere`	int	`0`	Number of spatial/positional dimensions appended to each vector. When > 0, enables P
`nWhen`	int	`0`	Number of temporal dimensions appended to each vector. When > 0, enables Temporal
`quantized`	bool	`false`	Whether input values are quantized/discrete.
`lexer`	string	`"word"`	Tokenization mode for text inputs. Common values in the current configs are `"word"` an

Note: InputSpace's `inputShape` uses the data's native dimension (e.g. 784 for MNIST, `inputLength` for text), while `outputShape` uses `nDim` from TheObjectEncoding. The LiftingLayer bridges the two.

Layers: Contains a `LiftingLayer` that projects raw input into the model's working dimensionality.

<PerceptualSpace>

Transforms lifted input into perceptual features via Pi layers (multiplicative interactions).

Parameter	Type	Default	Description
`nActive`	int	*required*	Number of active perceptual feature vectors (sparse subset selected in forward pass). Sh
`nDim`	int	`1`	Dimensionality of each perceptual vector. Set on TheObjectEncoding; not passed to the
`nVectors`	int	`0`	Codebook size (total vectors in the space). When > 0, enables vector quantization with
`invertible`	bool	`false`	Use a single invertible Pi layer instead of separate forward/reverse layers. When `true`, o
`hasAttention`	bool	`true`	Enable attention mechanism in this space.
`passThrough`	bool	`false`	Skip perceptual processing entirely; pass input through unchanged.

Layers: Pi layers — multiplicative: `y_j = b_j * prod_i(1 + W_ji * x_i)`. See Architecture.md.

<ConceptualSpace>

Transforms perceptual features into abstract concepts via Sigma layers (additive/linear).

Parameter	Type	Default	Description
`nActive`	int	*required*	Number of active concept vectors (sparse subset). For XOR: 3.
`nDim`	int	`1`	Dimensionality of each concept vector. Set on TheObjectEncoding; not passed to the Sp
`nVectors`	int	`0`	Codebook size (total vectors in the space).

Parameter	Type	Default	Description
`invertible`	bool	`false`	Use single invertible Sigma layer vs. separate forward/reverse layers (`sigma1`/`sigma2`).
`hasAttention`	bool	`false`	Enable attention in conceptual processing.
`hasNorm`	bool	`false`	Enable layer normalization in this space.

Layers: Sigma layers — additive: `y_j = b_j + sum_i(W_ji * x_i)`. See Architecture.md.

`<SymbolicSpace>`

Discrete symbolic representation — the information bottleneck between perception and output.

Parameter	Type	Default	Description
`nActive`	int	*required*	Number of active symbols. This is the full count; when reconstruction symbols are ena
`nDim`	int	`1`	Dimensionality of each symbol (typically 0 — symbols are zero-dimensional). Set on T
`nVectors`	int	= `nActive`	Codebook size (total vectors in the space).
`passThrough`	bool	`false`	Pass concepts through as symbols unchanged (no learned transformation). Typically t

Layers: None when `passThrough=true`. The bottleneck effect comes from the dimensionality constraint, not learned weights.

`<SyntacticSpace>`

Syntactic processing stage between symbols and output. Used when `symbolicOrder >= 1` to add an extra transformation layer. Future work will integrate generative grammar operations here.

Parameter	Type	Default	Description
`nActive`	int	`16`	Number of active word vectors (sparse subset selected in forward pass).
`nDim`	int	`1`	Dimensionality of each word vector (symbolic layer, so typically low-dimensional). Set on
`nVectors`	int	= `nActive`	Codebook size (total vectors in the space). Stored on TheObjectEncoding as `nWords`.

Layers: Currently a passthrough that reshapes symbols without transformation.

`<OutputSpace>`

Maps symbols to final predictions via linear layers.

Parameter	Type	Default	Description
`nActive`	int	*required*	Number of active output values. For XOR: 1 (single binary output).
`nDim`	int	`1`	Dimensionality of each output. Set on TheObjectEncoding; not passed to the Space cons
`nVectors`	int	= `nActive`	Codebook size (total vectors in the space). Defaults to `nActive` for OutputSpace.

Layers: Linear layers with `(bias, temp)` support for ergodic mode.

`<architecture><server>`

HTTP server settings (used by `serve.py`).

Parameter	Type	Default	Description
`host`	string	`"127.0.0.1"`	Server bind address.
`port`	int	`8001`	Server port.
`timeout`	int	`120`	Request timeout in seconds.

Example: XOR Configuration

```
<model>
  <architecture>
    <reconstruct>symbols</reconstruct>
    <reshape>true</reshape>
    <modelType>lm</modelType>
    <ergodic>false</ergodic>

    <data>
      <dataset>xor</dataset>
    </data>

    <training>
      <numEpochs>1000</numEpochs>
      <learningRate>0.01</learningRate>
      <autoload>false</autoload>
      <autosave>false</autosave>
    </training>
  </architecture>

  <InputSpace>
    <nActive>2</nActive>
    <nDim>1</nDim>
  </InputSpace>

  <PerceptualSpace>
    <nActive>4</nActive>
    <nDim>1</nDim>
    <nVectors>16</nVectors>
    <invertible>false</invertible>
    <hasAttention>false</hasAttention>
  </PerceptualSpace>

  <ConceptualSpace>
    <nActive>3</nActive>
    <nDim>1</nDim>
    <nVectors>16</nVectors>
    <invertible>false</invertible>
  </ConceptualSpace>

  <SymbolicSpace>
    <nActive>3</nActive>
```

```
    <passThrough>true</passThrough>
  </SymbolicSpace>

  <OutputSpace>
    <nActive>1</nActive>
    <nDim>1</nDim>
  </OutputSpace>
</model>
```

In this configuration:

- 2 binary inputs (`nActive=2`) are lifted, transformed through 4 perceptual and 3 conceptual active features
- PerceptualSpace and ConceptualSpace each have a codebook of 16 vectors (`nVectors=16`), selecting `nActive` via topk
- 3 symbols are produced: 1 for output prediction, 2 for reconstruction
- The reverse pass reconstructs the original 2 inputs from all 3 symbols
- Ergodic mode is off; training uses standard Adam with combined forward+reverse loss

---

Example: Embedding / Chatbot Configuration

```
<model>
  <architecture>
    <reconstruct>symbols</reconstruct>
    <modelType>embedding</modelType>
    <maskedPrediction>ARLM</maskedPrediction>

    <data>
      <dataset>text</dataset>
      <shardDir>data/fineweb</shardDir>
      <numShards>1</numShards>
      <maxDocs>10000</maxDocs>
      <minFrequency>0.00001</minFrequency>
    </data>

    <training>
      <numEpochs>1</numEpochs>
      <batchSize>1</batchSize>
      <learningRate>0.001</learningRate>
      <trainEmbedding>ARLM</trainEmbedding>
      <weightsPath>BasicModel.ckpt</weightsPath>
      <embeddingPath>BasicModel.kv</embeddingPath>
      <autoload>true</autoload>
      <autosave>true</autosave>
      <negSamples>64</negSamples>
    </training>
  </architecture>

  <InputSpace>
    <nActive>128</nActive>
    <nDim>100</nDim>
    <nWhere>2</nWhere>
    <nWhen>2</nWhen>
    <lexer>sentence</lexer>
```

```
    <quantized>true</quantized>
  </InputSpace>

  <!-- ... space definitions ... -->
</model>
```

Key points:

- `modelType=embedding` activates the embedding-based input pipeline alongside the neural model
- `trainEmbedding=ARLM` trains only the network layers; the codebook is detached so no gradients flow through it
- The three files partition model behaviour: **XML config** (architecture), **`.kv` embedding** (codebook), **`.ckpt` weights** (model layers)
- `weightsPath` stores the neural model checkpoint; `embeddingPath` stores the word vectors
- `minFrequency` gates vocabulary admission: words are buffered until their frequency ratio exceeds this threshold